



Vue.js 금융 대시보드 개발

3단계 구현 가이드

Vite + Vue 3로 만드는 실무형 SPA 프로젝트 완전 정복

STEP 1



기초 & 라우팅

Vite 프로젝트 생성
Vue Router 설정
임시 로그인 & API 맛보기

STEP 2



UI & 시각화

Vuetify 디자인 적용
Chart.js 시계열 차트
실제 서비스 같은 UI 구현

STEP 3



인증 & 상태관리

Firebase Auth + Firestore
Pinia 스토어 관리
Router Guard & Toast



© 2025 Finance Dashboard Project. All rights reserved.

OVERVIEW

학습 로드맵 & 핵심 원칙

1단계: 기초 구현 & 라우팅

2단계: UI 디자인 & 시각화

3단계: 인증 & 상태관리

비교 요약 및 마무리

1

1단계: 기능 중심 (Minimal) Vite + Router + Axios

금지 항목: Firebase, Pinia, Mitt, Vuetify, Chart.js 오직 라우팅 구조와 임시 로그인 로직, 기본 API 호출에만 집중합니다.

No Auth

No UI Lib

2

2단계: 디자인 & 차트 (Visual) Vuetify + Chart.js

1단계 코드 위에 디자인 시스템과 데이터 시각화를 입힙니다. 인증/저장은 여전히 더미 상태를 유지하며 UI 완성도에 집중합니다.

Vue-Chartjs

Date-fns

3

3단계: 실제 로직 완성 (Real) Firebase + Pinia + Mitt

이제서야 진짜 인증과 전역 상태관리를 도입합니다. Router Guard, Global Toast, Firestore 저장이 추가됩니다.

Auth

Firestore

Store

Introduction

Step 1

Step 2

Step 3

Summary

STEP 1

기초 구현& 라우팅

Vite 프로젝트를 생성하고

Vue Router로 기본 골격을 잡는 단계입니다.

🎯 핵심 목표

Vite + Vue 3 프로젝트 생성

SPA 라우팅 구조 설계

임시 로그인(localStorage) 구현

Axios로 외부 API 호출 맛보기



Vite + Vue 3

최신 프론트엔드 빌드 도구와 Vue 3
Composition API 기반 환경 설정



Vue Router

/login, /register, /dashboard 페이지 이동 및
SPA 구조 이해



Axios API

비동기 데이터 통신 기초 및 외부 금융 데이터 연동
실습



임시 인증

복잡한 백엔드 없이 localStorage를 활용한
간단한 로그인 흐름



1단계 제한 사항 (아직 사용하지 않음)

Firebase

Pinia

Vuetify

Chart.js

mitt

Step 1: 기초

Step 2: UI

Step 3: 인증



STEP 1. SETUP

프로젝트 생성 및 패키지 설치

bash - 80x24

1. Vite 프로젝트 생성 (Vue 3 선택)

```
$npm create vite@latest finance-dashboard
```

✓ Select a framework: Vue

✓ Select a variant: JavaScript

2. 프로젝트 폴더 이동 및 의존성 설치

```
$cd finance-dashboard
```

```
$npm install
```

3. 1단계 핵심 라이브러리 추가

```
$npm install vue-router axios
```

✓ 설치 패키지

vue-router 페이지 이동(라우팅)을 위한 공식 라이브러리

axios 환율 및 코인 정보를 가져올 HTTP 클라이언트

⊘ 아직 설치하지 않음

Firebase, Pinia, Vuetify, Chart.js 등은 2, 3단계에서 순차적으로 설치합니다. 지금은 기본 기능 구현에 집중하세요.

Step 1: 기초

Step 2: UI

Step 3: 인증



main.js (1단계)

```
1 import {createApp} from 'vue'
2 import App from './App.vue'
3 import router from './router' // 1단계: 라우터 불러오기
4
5 import './assets/main.css'
6
7 const app = createApp(App)
8
9 app.use(router) // Vue 인스턴스에 라우터 등록
10
11 app.mount('#app')
```

i 핵심 포인트

Vite 프로젝트 생성 직후에는 라우터가 없으므로, npm install vue-router 후 직접 import하고 app.use()로 등록해야 합니다.

⚠ 주의사항

이 단계에서는 아직 createPinia()나 createVuetify()를 사용하지 않습니다. 가장 기본적인 Vue + Router 구조만 잡습니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



라우터 구성 (기본 버전)

src/router/index.js

```
1 import {createRouter, createWebHistory} from 'vue-router'
2 import LoginView from '../views/LoginView.vue'
3 import RegisterView from '../views/RegisterView.vue'
4 import DashboardView from '../views/DashboardView.vue'
5
6 const routes = [
7   {path: '/', redirect: '/login'},
8   {path: '/login', component: LoginView},
9   {path: '/register', component: RegisterView},
10  {path: '/dashboard', component: DashboardView}
11 ]
12
13 const router = createRouter({
14   history: createWebHistory(),
15   routes
16 })
17
18 export default router
```

1 기본 라우팅 구조

3개의 핵심 뷰(Login, Register, Dashboard)를 등록하고, 루트 경로('/') 접근 시 로그인 페이지로 리다이렉트합니다.

라우트 정의 (Routes)

/login → LoginView

/register → RegisterView

/dashboard → DashboardView



참고: 가드(Guard) 없음

1단계에서는 Navigation Guard를 사용하지 않습니다. 즉, 로그인하지 않아도 URL만 알면 대시보드에 접근 가능한 "장난감" 상태입니다.



App.vue (1단계 - 최소 골격)

src/App.vue

```
1 <template>
2 <div class="app">
3 <h1 class="app-title">Finance Dashboard (Step 1) </h1>
4
5 <nav class="nav">
6 <RouterLink to="/login">로그인 </RouterLink>
7 <RouterLink to="/register">회원가입 </RouterLink>
8 <RouterLink to="/dashboard">대시보드 </RouterLink>
9 </nav>
10
11 <main class="main">
12 <RouterView />
13 </main>
14 </div>
15 </template>
16
17 <script setup></script>
18
19 <style scoped>
20 .app { max-width: 960px; margin: 0 auto; padding: 24px; }
21 .nav { display: flex; gap: 12px; }
22 .main { border: 1px solid #e5e7eb; border-radius: 8px; }
```

Navigation RouterLink 사용

일반 `<a>` 태그 대신 Vue Router의 `<RouterLink>`를 사용하여 페이지 새로고침 없는 SPA 네비게이션을 구현합니다.

Rendering RouterView 배치

현재 URL에 매칭되는 컴포넌트(Login, Register, Dashboard 등)가 `<RouterView />` 위치에 렌더링됩니다.

Design 최소한의 CSS

1단계에서는 Vuetify 같은 UI 라이브러리 없이, 기본적인 CSS만 사용하여 레이아웃을 잡습니다. (중앙 정렬, 간격 등)

Step 1: 기초

Step 2: UI

Step 3: 인증



LoginView (임시 로그인)



src/views/LoginView.vue

```
LoginView.vue

<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'

const email = ref('')
const password = ref('')

const router = useRouter()

const handleLogin = () => {
  // 1단계: 실제 인증 없음, 바로 대시보드로
  router.push('/dashboard')
}
</script>
```

구현 포인트

1단계에서는 Firebase '로그인 상태'를 흉내 냅니다.
v-model을 사용해 입력값을 받고, 버튼 클릭 시 대시보드로 이동합니다.

의도

"보안"보다는 라우팅 흐름(Router Flow)을 먼저 이해시키는 것이 목적입니다.
진짜 인증은 3단계에서 Firebase로 교체됩니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



```
RegisterView.vue <script setup>

<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'

const email = ref('')
const password = ref('')
const passwordConfirm = ref('')

const router = useRouter()

const handleRegister = () => {
  if (password.value !== passwordConfirm.value) {
    alert('비밀번호가 일치하지 않습니다.')
    return
  }

  // 1단계: 실제 Firebase 등록 없이, 그냥 안내 후 로그인 페이지로 이동
  alert('1단계 데모: 회원가입이 완료되었다고 가정하고 로그인 화면으로 이동합니다.')
  router.push('/login')
}
</script>
```

✓ 목표 확인

실제 백엔드 연동 전, UI 레이아웃과 라우터 이동이 정상적으로 동작하는지 확인하는 것이 목표입니다.

i 참고

Firebase Auth를 이용한 실제 회원가입 로직은 3단계에서 `userStore.register()` 호출로 대체됩니다.



DashboardView (API 호출)

src/views/DashboardView.vue

DashboardView.vue

```

<script setup>
import { ref, onMounted } from 'vue'
import { fetchUsdToKrwAndOthers } from '../services/forexApi'
import { const forexLoading: Ref<boolean> } from '../services/cryptoApi'

const forexLoading = ref(false)
const forexLoading = ref(false)
const forexError = ref('')

const cryptoPrices = ref({})
const cryptoLoading = ref(false)
const cryptoError = ref('')

onMounted(async () => {
  // 환율
  forexLoading.value = true
  try {
    const data = await fetchUsdToKrwAndOthers()
    forexRates.value = data.rates
  } catch (err) {
    forexError.value = '환율 정보를 가져오는데 실패했습니다.'
    console.error(err)
  } finally {
    forexLoading.value = false
  }
}

```

⚡ 1단계 특징

디자인 라이브러리(Vuetify)나 차트 없이, <pre> 태그에 JSON 원본 데이터를 그대로 출력하여 "데이터 연결" 자체에 집중합니다.

☁ Axios 활용

무료 API(Frankfurter, CoinGecko)를 axios.get()으로 호출합니다.
CORS나 인증 키 설정이 필요 없는 오픈 API를 사용하여 실습 복잡도를 낮춥니다.

Step 1: 기초

Step 2: UI

Step 3: 인증

UI 디자인& 데이터 시각화

1단계 기능 위에 Vuetify로 디자인을 입히고
Chart.js로 데이터를 시각화하는 단계입니다.

🎯 핵심 목표

Vuetify 설치 및 레이아웃 적용
금융 정보 카드(Forex/Crypto) 구현
Chart.js 및 vue-chartjs 설치
시계열 라인 차트(History) 구현



Vuetify 3

Material Design 기반의 완성도 높은 UI 컴포넌트 프레임워크



Chart.js

HTML5 Canvas 기반의 강력한 데이터 시각화 라이브러리



UI Components

재사용 가능한 카드(Card) 및 보드(Board) 컴포넌트 설계



Date Adapter

date-fns 어댑터를 활용한 효율적인 시계열 데이터 처리



2단계 제한 사항 (아직 사용하지 않음)

Firebase

Pinia

mitt

실제 인증

02



STEP 2. UI & CHARTS

2단계 추가 패키지 설치

bash - 80x24

1. 차트 라이브러리 및 Vue 래퍼 설치

```
$npm install chart.js vue-chartjs
```

2. 시계열 데이터 처리를 위한 어댑터 설치

```
$npm install chartjs-adapter-date-fns date-fns
```

주요 라이브러리

Vuetify Material Design 기반의 Vue UI 프레임워크

Chart.js 강력한 데이터 시각화 및 차트 구현

date-fns 차트의 시간축(X축) 처리를 위한 날짜 유틸리티

아직 설치하지 않음

Firebase, Pinia, mitt는 3단계에서 설치합니다. 현재는 UI와 시각화에 집중합니다.

Step 1: 기초

Step 2: UI & Chart

Step 3: 인증

Vite 프로젝트에 Vuetify 설정

1 NPM INSTALL

```
➔ npm install vuetify
➔ npm install vite-plugin-vuetify
```

2 MAIN ENTRY (SRC/MAIN.JS)

```
// components, directives: Vuetify 컴포넌트를 전역 등록
// theme: 라이트/다크 테마 정의
const vuetify = createVuetify({
  components,
  directives,
  theme: {
    defaultTheme: 'light',
    themes: {
      light: { dark: false },
      dark: { dark: true }
    }
  }
})
```



Vite Config 설정

vite.config.js

```
import vuetify from 'vite-plugin-vuetify'
export default defineConfig({
  plugins: [
    vue(),
    vuetify({ autoImport: true }) // 자동 임포트
  ]
})
```

vite-plugin-vuetify를 사용하면 컴포넌트를 일일이 import 하지 않아도 필요한 것만 자동으로 불러와(Tree Shaking) 번들 크기를 최적화합니다.

✓ 설정 체크리스트



테마 시스템 (Theme)

createVuetify 옵션에서 light/dark 테마를 정의합니다. 나중에 theme.global.name 값을 변경하여 다크모드를 전환할 수 있습니다.



App.vue – Vuetify 레이아웃 적용

```
src/App.vue<template>
  <v-app>
    <v-app-bar app>
      <v-app-bar-title>Finance Dashboard</v-app-bar-title>
    </v-app-bar>

    <v-main>
      <v-container fluid>
        <router-view />
      </v-container>
    </v-main>
  </v-app>
</template>
```

1 Vuetify 기본 골격

Vuetify 앱의 필수 구조를 적용했습니다.

<v-app>: 최상위 래퍼 (필수)

<v-main>: 콘텐츠 영역 자동 조절

<v-container>: 반응형 패딩 적용

☰ 상단바 (App Bar)

🎨 Indigo 테마를 적용하여 금융 앱 느낌 강조

↔ <v-spacer />로 타이틀과 메뉴 양쪽 정렬

🔗 기존 <nav> 대신 상단바에 링크 배치



이 단계에서는 아직 로그인 상태에 따른 메뉴 변경은 구현하지 않습니다.
(3단계에서 적용)

Step 1: 기초

Step 2: UI & 차트

Step 3: 인증



STEP 2. UI STRUCTURE

대시보드 레이아웃 구조

DASHBOARDVIEW.VUE TEMPLATE STRUCTURE

v-col (md=6)

<ForexCard />

\$ 주요 환율 정보 카드

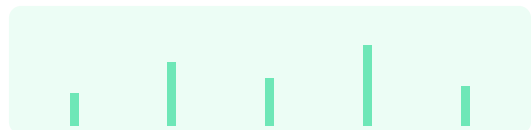
v-col (md=6)

<CryptoCard />

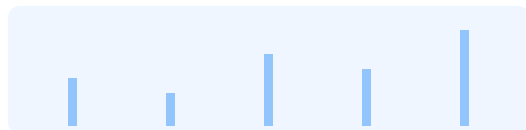
฿ 코인 시세 정보 카드

MyChartBoard.vue

<MyChartBoard />



<CryptoLineChart
asset="btc" />



<CryptoLineChart
asset="eth" />

템플릿 구현 (Vuetify Grid)

```
<template>
  <div class="dashboard-page">
    <header class="dashboard-header">
      <h1 class="dashboard-title">Finance Dashboard</h1>
      <p class="dashboard-subtitle">
        환율과 암호화폐 시세를 한눈에 확인하는 대시보드
      </p>
    </header>

    <div class="dashboard-grid">
      <div class="dashboard-card-wrapper">
        <ForexCard />
      </div>
      <div class="dashboard-card-wrapper">
        <CryptoCard />
      </div>
    </div>
  </div>
</template>
```

Vuetify의 Grid System (v-row, v-col)을 사용하여 반응형 레이아웃을 구성합니다. 상단에는 정보 카드를 배치하고, 하단에는 차트 보드 컴포넌트를 위치시킵니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



ForexCard / CryptoCard 구현

\$ ForexCard.vue

Frankfurter API 사용

USD 기준 KRW, EUR, JPY 환율 표시

🪙 CryptoCard.vue

CoinGecko API 사용

BTC, ETH 현재 가격 (USD)

💎 공통 구현 사항 (Vuetify)

v-card 컴포넌트로 카드 UI 구성

loading / error 상태에 따른 UI 처리

```
src/components/ForexCard.vue

<script setup>
import { ref, onMounted } from 'vue'
import { fetchUsdToKrwAndOthers } from '../services/forexApi'

const rates = ref({})
const loading = ref(false)
const error = ref('')
const lastDate = ref('')

onMounted(async () => {
  loading.value = true
  error.value = ''

  try {
    const data = await fetchUsdToKrwAndOthers()
    // data: { amount, base, date, rates: { KRW: ..., EUR: ... }
    rates.value = data.rates
    lastDate.value = data.date
  } catch (err) {
    console.error(err)
    error.value = '환율 정보를 가져오는데 실패했습니다.'
  } finally {
    loading.value = false
  }
})
</script>
```




CryptoLineChart.vue

```
const loadData = async () => {
  loading.value = true
  error.value = ''
  try {
    const id = props.asset === 'btc' ? 'bitcoin' : 'ethereum'
    const data = await fetchCryptoHistory(id, days.value)
    prices.value = data
  } catch (err) {
    console.error('[CryptoLineChart.loadData]', err)
    error.value = '차트 데이터를 불러오는데 실패했습니다.'
  } finally {
    loading.value = false
  }
}
```

```
const chartOptions = {
  responsive: true,
  maintainAspectRatio: false,
  scales: {
    x: {
      type: 'time',
      time: { unit: 'day' }
    },
    y: {
      ticks: {
        callback: (value) => `$$${value}`
      }
    }
  },
  plugins: {
    legend: { display: false }
  }
}
```

✓ 핵심 컴포넌트

chartjs-adapter-date-fns가 있어야 x축을 시간(time) 타입으로 설정 가능합니다.
vue-chartjs의 Line 컴포넌트에 data와 options를 prop으로 전달합니다.

📖 기능 요약

7/30/90일 범위 선택 버튼을 통해 기간별 시세 데이터를 불러오고, 차트 데이터를 갱신합니다.
2단계에서는 로컬 상태나 임시 스토어만 사용하여 데이터를 관리합니다.

Step 1: 기초

Step 2: UI & Charts

Step 3: 인증



```
<template>

  <div class="dashboard-card board">
    <v-row>
      <v-col cols="12" md="6">
        <CryptoLineChart asset="btc" title="Bitcoin (BTC)" />
      </v-col>
      <v-col cols="12" md="6">
        <CryptoLineChart asset="eth" title="Ethereum (ETH)" />
      </v-col>
    </v-row>
  </div>
</template>
```

📁 컴포넌트 구조

v-row와 v-col을 사용해 반응형 그리드를 구성하고, 동일한 차트 컴포넌트를 props만 다르게 하여 재사용합니다.

🔗 스토어 연동 (3단계 준비)

loadPrefs()는 2단계에선 로컬 상태만 다루지만, 3단계에선 Firestore와 연동될 중요한 연결 고리입니다.

Step 1: 기초

Step 2: UI & 차트

Step 3: 인증



STEP 3. AUTHENTICATION & STATE

인증 및 상태 관리 도입



Firebase

실제 이메일/비밀번호 인증(Auth)과 사용자 데이터 저장을 위한 Firestore 데이터베이스를 연동합니다.



Pinia

로그인 사용자 정보(User)와 차트 설정(Preferences)을 전역에서 관리하는 상태 관리 라이브러리입니다.



mitt

컴포넌트 어디서든 간편하게 알림 메시지(Toast)를 띄우기 위한 경량 이벤트 버스입니다.



bash - Step 3 Installation

3단계 핵심 패키지 설치 (이전 단계에는 없던 것들)

```
$ npm install firebase pinia mitt
```

Step 1: 기초

Step 2: UI

Step 3: 인증

3단계 > 패키지 설치 및 개요



firebase.js (3단계 신규 생성)

```
1 import { initializeApp } from 'firebase/app'
2 import { getAuth } from 'firebase/auth'
3 import { getFirestore } from 'firebase/firestore'
4
5 // ♦ Firebase 콘솔에서 발급받은 설정값 (복사 & 붙여넣기)
6 const firebaseConfig = {
7   apiKey: 'AIzaSy...',
8   authDomain: '...',
9   projectId: '...',
10  /* ... 기타 설정값 ... */
11 }
12
13 const app = initializeApp(firebaseConfig)
```

☰ 3단계 핵심 변경

1, 2단계에서는 존재하지 않았던 Firebase 연결 파일을 생성합니다.
앱 내 어디서든 이 파일의 auth, db를 import하여 사용합니다.

⚙️ 설정 팁

Firebase Console > Project Settings에서 Web App을 추가하고 firebaseConfig 객체를 복사해오세요. Auth(Email/Password)와 Firestore 활성화가 필요합니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



Pinia + mitt 연결 + GlobalToast

```
src/main.js (3단계)

import { useUserStore } from './stores/userStore'
import { emitter } from './plugins/eventBus'

const vuetify = createVuetify()

const app = createApp(App)
const pinia = createPinia()

app.use(pinia)
app.use(router)
app.use(vuetify)

// Pinia 스토어 초기화 (Auth 상태 감시)
const userStore = useUserStore()
userStore.initAuthListener()

// mitt 전역 등록
app.config.globalProperties.$bus = emitter

app.mount('#app')
```

1 Pinia (상태 관리)

Vuex의 후속인 Pinia를 도입하여 전역 상태(사용자 정보, 차트 설정 등)를 효율적으로 관리합니다. createPinia()로 인스턴스를 생성하고 앱에 플러그인으로 등록합니다.

2 mitt (이벤트 버스)

Vue3에서 제거된 Event Bus 기능을 대체하는 초경량 라이브러리입니다.

```
app.config.globalProperties.$bus = bus
```

이렇게 등록하면 컴포넌트 어디서든 \$bus로 접근 가능합니다.



GlobalToast 활용

모든 페이지에서 공통으로 사용하는 알림(로그인 성공/실패 등)을 처리하기 위해 App.vue에 한 번만 배치하고, 이벤트 버스로 제어합니다.



userStore 구현 (Firebase Auth)



src/stores/userStore.js

userStore.js

```
import { defineStore } from 'pinia'
import { auth } from '../firebase'
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged
} from 'firebase/auth'
import router from '../router'
```

// 🔥 이 줄 추가

11 references

```
export const useUserStore = defineStore('user', {
  state: () => ({
    user: null,
    loading: false,
    error: ''
  }),
```

initAuthListener를 App.vue나 main.js에서 최초 1회 호출하여, 새로고침 후에도 Firebase 로그인 상태가 Pinia 스토어에 유지되도록 합니다.

Pinia에서는 actions 내부에서 비동기(Firebase API) 처리 후 직접 state(this.user)를 변경하면 됩니다.



chartPrefStore.js

```
export const useChartPrefStore = defineStore('chartPrefs', {
  state: () => ({
    btcDays: 30,
    ethDays: 30,
    loading: false,
    error: ''
  }),
  actions: {
    async loadPrefs() {
      this.loading = true
      this.error = ''
      try {
        const userStore = useUserStore()
        if (!userStore.user) return

        const ref = doc(db, 'chartPrefs', userStore.user.uid)
        const snap = await getDoc(ref)
        if (snap.exists()) {
          const data = snap.data()
          if (typeof data.btcDays === 'number') this.btcDays = data.btcDays
          if (typeof data.ethDays === 'number') this.ethDays = data.ethDays
        }
      }
    }
  }
})
```

🔥 Firestore 연동

chartPrefs/{uid} 경로에 사용자별 설정을 저장합니다. 로그인 시 loadPrefs()를 호출하여 이전에 보던 차트 기간을 복원합니다.

🔗 Store 간 통신

차트 설정 스토어에서 useUserStore()를 가져와 현재 로그인된 사용자의 UID에 접근하는 패턴을 확인하세요.

Step 1: 기초

Step 2: UI

Step 3: 인증



router/index.js (3단계)

```
router.beforeEach((to, from, next) => {
  const isLoggedIn = !!auth.currentUser // 🔥 핵심: 언제나 정확함
  console.log('auth.currentUser ', auth.currentUser)
  // 1) 로그인 필요한데 로그인 안됨
  if (to.meta.requiresAuth && !isLoggedIn) {
    return next('/login')
  }

  // 2) 로그인된 상태에서 /login 또는 /register 접근
  if (to.meta.guestOnly && isLoggedIn) {
    return next('/dashboard')
  }

  next()
})
```

**Navigation Guard 로직**

requiresAuth 메타 필드가 있는 라우트는 비로그인 접근을 차단하고, guestOnly는 로그인한 사용자가 불필요하게 로그인 페이지에 접근하는 것을 막습니다.

**실무 주의사항**

새로고침 시 Firebase SDK 초기화 전에 가드가 실행되면 currentUser가 null일 수 있습니다. 실무에선 초기화 완료를 기다리는 로직이 추가로 필요합니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



App.vue - 상단바 및 전역 토스트

```
src/App.vue

<div v-if="isLoggedIn" class="d-flex align-center">
  <span class="user-email">
    {{ userEmail }}
  </span>
  <v-btn
    size="small"
    variant="outlined"
    color="white"
    class="ml-3"
    @click="handleLogout"
  >
    로그아웃
  </v-btn>
</div>

<div v-else class="d-flex align-center">
  <v-btn
    size="small"
    variant="text"
    color="white"
    @click="goLogin"
  >
    로그인
```

Step 1: 기초

Step 2: UI

Step 3: 인증

1 조건부 렌더링 (v-if / v-else)

user 객체 존재 여부에 따라 상단 메뉴가 변경됩니다. 로그인 시 이메일 + 로그아웃 버튼을, 비로그인 시 로그인/회원가입 링크를 노출합니다.

UI/UX 개선 포인트

로그인 정보 표시: 현재 접속 중인 사용자 이메일을 상단바에 표시하여 상태를 명확히 함

전역 토스트: GlobalToast를 최상위에 배치하여 어디서든 알림 메시지 노출 가능

로그아웃: 버튼 클릭 시 Store 액션 호출 및 토스트 알림 후 리다이렉트



Pinia 상태 연결

useUserStore의 반응형 상태(State)를 computed로 연결하여, 로그인/로그아웃 시 화면이 즉시 갱신되도록 처리합니다.



Login/Register (Firebase 버전)

src/views/LoginView.vue (3단계)

```

const { appContext } = getCurrentInstance()
const bus = appContext.config.globalProperties.$bus

const handleLogin = async () => {
  try {
    await userStore.login(email.value, password.value)
    bus.emit('toast', { type: 'success', message: '로그인 성공!' })
    router.push('/dashboard')
  } catch (err) {
    bus.emit('toast', { type: 'error', message: '로그인 실패' })
    console.error(err)
  }
}

```

1 더미 코드 완전 제거

실제 비동기 인증 로직으로 대체합니다.

구현 핵심 포인트

- 1 userStore 활용**
컴포넌트에서 직접 Firebase SDK를 호출하지 않고, Pinia Store의 액션을 통해 처리 (관심사 분리)
- 2 예외 처리 & UX**
try-catch로 에러를 잡고, \$bus.emit으로 전역 토스트 메시지를 띄워 사용자 경험 향상

i 회원가입(RegisterView)도 동일한 패턴으로 userStore.register()를 호출하여 구현합니다.

Step 1: 기초

Step 2: UI

Step 3: 인증

단계별 구현 내용 비교 요약

1 기초 구현 (Blue)

Vite Vue Router Axios 임시 로그인: localStorage로
단순 처리 (보안 없음) API 맛보기: 환율/코인 정보를
텍스트(JSON)로만 출력 UI 라이브러리, 차트,
상태관리 없음

2 UI & 시각화 (Green)

Vuetify Chart.js vue-chartjs 대시보드 UI: 카드
배치를 통한 레이아웃 구성 시각화: CryptoLineChart
등으로 시계열 데이터 표현 로그인은 여전히 임시 상태
유지

3 인증 & 상태 (Red)

Firebase Pinia mitt 실제 인증: Firebase Auth 기반
전역 상태: User 정보 및 차트 설정을 Pinia로 관리
보안 강화: Router Guard로 접근 제어
(requiresAuth)

Step 1: 기초

Step 2: UI

Step 3: 인증

🌐 전체 과정 완료

CONCLUSION

최종 정리 & Q&A

Vue.js 금융 대시보드 3단계 구현 가이드



핵심 패턴 요약

- 단계적 확장(Incremental): 기초 라우팅 → UI 시각화 → 인증/상태 도입 순으로 복잡도 관리
- 관심사 분리: View(화면)와 Store(데이터/로직)의 명확한 역할 구분
- 컴포넌트 재사용: CryptoLineChart 등 공통 컴포넌트 활용



실습 팁 & 다음 단계

API 관리: 무료 API(Frankfurter, CoinGecko) 사용량 제한 주의

디버깅 포인트: 새로고침 시 Firebase Auth 초기화 타이밍 문제 (onAuthStateChanged)

배포(Deploy): Vercel/Netlify로 배포하고 환경변수 설정하기



Q & A

구현 과정이나 코드에 대해
궁금한 점이 있다면 질문해주세요.

#Vue3

#Firebase

#Pinia

감사합니다!

Vue.js 금융 대시보드 프로젝트 실습 완료